Video 4 – Error and Noise

Video Contents

Video Synopsis

Non-linear Transformation **03:36 – 12:35**

The non-linear transformation introduced in lecture 3 is formalised somewhat: points **x** = $(x_1, x_2, \cdots, x_d)$ (where d indicates the dimensionality of the input space X), are transformed to points **z** = $(z_1, z_2, \cdots, z_{\tilde{d}}) = \Phi(\boldsymbol{x})$ where Φ (capital phi) is the function that transform **x** into **z** and $\tilde{d}$ (d tilde) indicates that the dimension of the Z-space is likely different from the X space as it depends on the chosen transformation, which is entirely up to you. The output Y is not transformed and remains the same! Once the transformation is performed, the transformed data can be used in a linear model (such as linear regression). To indicate that this linear model now uses non-linearly transformed data as inputs, the weights are also indicated with a tilde: $\tilde{\boldsymbol{w}}$. In case of linear classification, this would result in a final hypothesis:

$$g(x) = \text{sign}(\tilde{\boldsymbol{w}}^T \boldsymbol{z}) = \text{sign}\big(\tilde{\boldsymbol{w}}^T \Phi(\boldsymbol{x})\big)$$

Error Measures **12:35 – 23:00**

An Error Measure is introduced as a measure that tells something about how alike the two arguments to the error measure are. In most cases the Error Measure is an average across the data set of a 'point-wise' error measure. As point-wise error measure, the squared error: $e\big(h(x), f(x)\big) = (h(x) - f(x))^2$ and the binary error e(h(x),f(x))= $[h(x) \neq f(x)]$ are formally introduced. In the latter, the square brackets indicate that the error function will evaluate to 1 if the argument is true and 0 otherwise. To now obtain the average of this point-wise error across the whole training set (i.e. the in-sample error E_{in}), you evaluate the point-wise error at each point, sum the result and divide by N: $E_{in}(h) = \frac{1}{N}\sum_{n=1}^{N} e\big(h(x_n), f(x_n)\big)$. A similar approach is not possible for the out-of-sample error as we don't know how many of those points we have or what their value is, so we need to be content with a more general equation for the out-of-sample error: $E_{out}(h) = \mathbb{E}_x[e(h(x), f(x))]$, i.e. the expected value (over the whole data space) of the point-wise error measure. With the error measure now properly defined, it is added to the learning diagram and the important observation is made that **the data points that will be drawn from the input data space to calculate the in-sample error, have to be drawn with the same probability distribution as those that were drawn from the input data space for training.** If this is not the case, the theory that we are building to prove that learning is possible, becomes null and void.

Choosing an Error measure ***23:00 – 36:14***

Using two examples Prof. Abu-Mostafa shows that choosing an error measure is ideally left up to the customer. Suppose that you use a finger print system to check if people in a shop have loyalty cards. Identifying a fingerprint as from a loyalty customer if this is not the case (false positive) carries a relatively small cost: it will only cost the shop the discount that this customer then will receive. On the other hand, a false negative in which the fingerprint of a loyalty customer is not recognised can be very costly as it may chase the customer to another shop! Compare this to a fingerprint system to get access to the CIA HQ: a false positive would now be disastrous as it

would allow an unauthorized person access, whereas a false negative is acceptable, as an employer will simply try again. Unfortunately it is not always possible to choose an error measure that takes the application into account as described and often we resort to plausible or simply convenient (from an algorithmic perspective) error measures. Finally, the error measure is added to the learning diagram with links to both the learning algorithm and the final hypothesis as the error measure is both used during training and during evaluation of the training performance.

Noisy Targets **36:14 – 47:58**

Prof. Abu-Mostafa explains that, even though we have been speaking of a target function f(x) so far, we can seldom strictly speak of a function as, due to noise, the output for a given x can be a multitude of values. For this reason the relationship between input x and output y is reformulated as a probability distribution: P(y|x), which, with the probability of picking a certain x: P(x) yields the joint distribution P(x)P(y|x) as the process that generates our training data. With this definition, the unknown target function in the learning diagram can be replaced by the unknown target distribution to make the learning diagram complete. Finally, the point is made that P(x)P(y|x) is often abbreviated in literature as the joint probability distribution P(x,y). Whilst this is mathematically absolutely correct, it is important that we do not want to learn P(x,y). We only want to learn P(y|x) as this is the unknown target distribution. P(x) can be seen as a weighting vector that weighs the individual importance of each x in determining P(x) based on the likelihood of encountering that particular x 'in the wild'.

Introduction to the Theory of Learning ***47:58 – 59:19***

Prof. Abu-Mostafa introduces the following lectures on the Theory of Learning by stating that we need to know two things to determine that learning is possible:

1. We need to know that in-sample error and out-of-sample error will be similar:$E_{in}(g) \approx E_{out}(g)$
2. We need to know that E_{in} will likely be small: $E_{in}(g) \approx 0$ as, with point 1, we can then show that the out-of-sample error will be small and that learning was indeed possible.

An important observation is made about what 'small' means in terms of E_{in}. Often small will be close to 0, but for example for financial forecasting, small could be an error just below .50 as doing better than random in, e.g., 3% of cases could already make you a lot of money.

Finally, a very important principle is very quickly introduced: in general one can say that, as the complexity of your model increases, your in-sample error will go down. This is due to the fact that more complex models are more flexible to do well on each and every point in the training data set. However, the trade-off is that more complex models tend to result in a bigger difference between in-sample and out-of-sample error, and this tends to hurt our reliance on using the likeness of E_{in} and E_{out} to calculate E_{in} and use its value to predict E_{out}.